Task 1 – Python + FastAPI

Books CRUD REST API (FastAPI, Pydantic, SQLite)

1. Overview

This project is a CRUD REST API for managing books. It supports creating, reading, updating and deleting records. Request data is validated with Pydantic, records are stored in an SQLite database, and the API was tested through Swagger UI.

Component	Technology
Web framework	FastAPI
Validation	Pydantic
Database	SQLite (sqlite3 module)
Server	Uvicorn

2. Project structure

```
fastapi-crud/
  app/
    main.py       - API routes (endpoints)
    schemas.py    - Pydantic models (validation rules)
    crud.py       - database operations (SQL)
    database.py   - SQLite connection and table creation
  requirements.txt
  README.md
```

3. How to run

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Open http://127.0.0.1:8000/docs in a browser to use Swagger UI.

4. API endpoints

Method	Endpoint	Description	Success	Errors
POST	/books/	Create a book	201	409, 422
GET	/books/	List all books	200	-
GET	/books/{id}	Get one book by ID	200	404
PUT	/books/{id}	Update a book	200	404, 409, 422
DELETE	/books/{id}	Delete a book	204	404

Error codes: 404 = book not found, 409 = duplicate ISBN, 422 = invalid input (Pydantic validation).

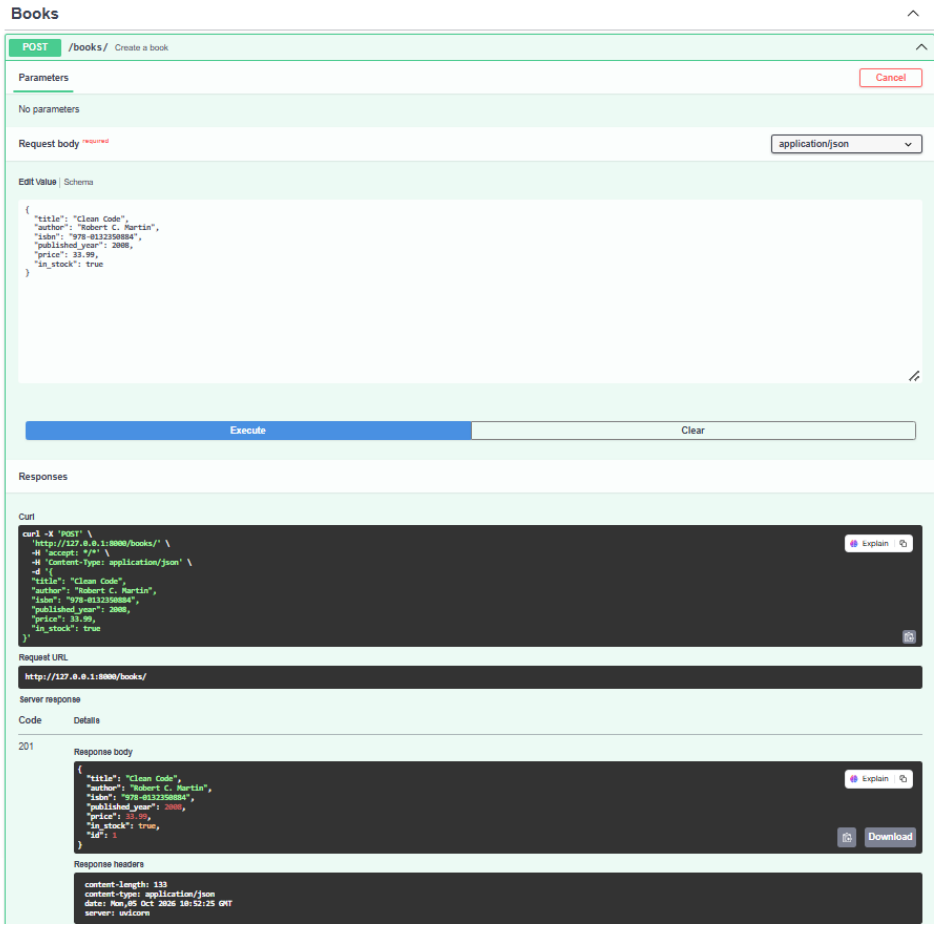
5. API testing results (Swagger UI)

Each test below was run in Swagger UI at /docs. The screenshots show the request and the server response.

1. POST /books/ – Create a book

A new book was created with a valid JSON body.

Expected result: 201 Created, the book is returned with an auto-generated id.



2. GET /books/ – List all books

All books in the database were fetched.

Expected result: 200 OK with a list of books. Response received:

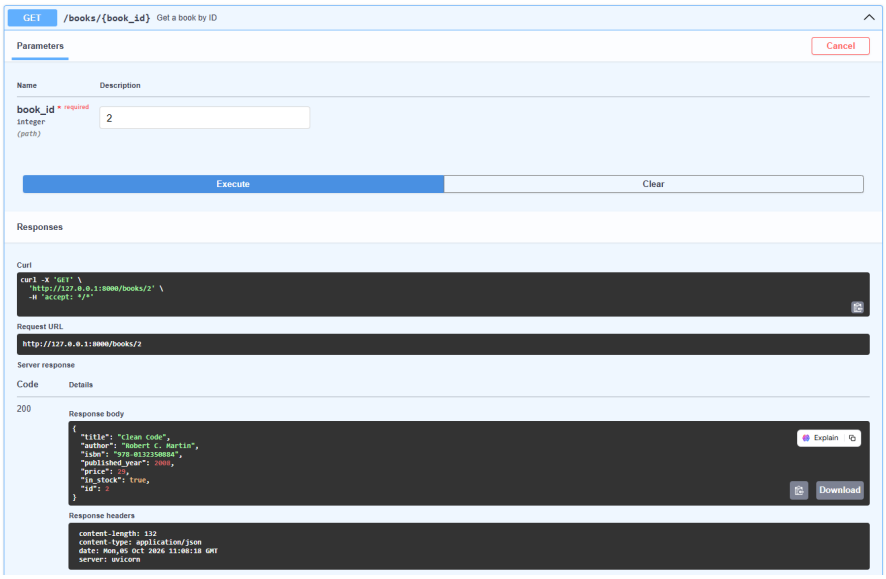

```
[
  {
    "title": "Clean Code",
    "author": "Robert C. Martin",
    "isbn": "978-0132350884",
    "published_year": 2008,
    "price": 33.99,
    "in_stock": true,
    "id": 1
  }
]
```


Status: 200 OK

3. GET /books/{id} – Get a book by ID

The book with id 2 was requested.

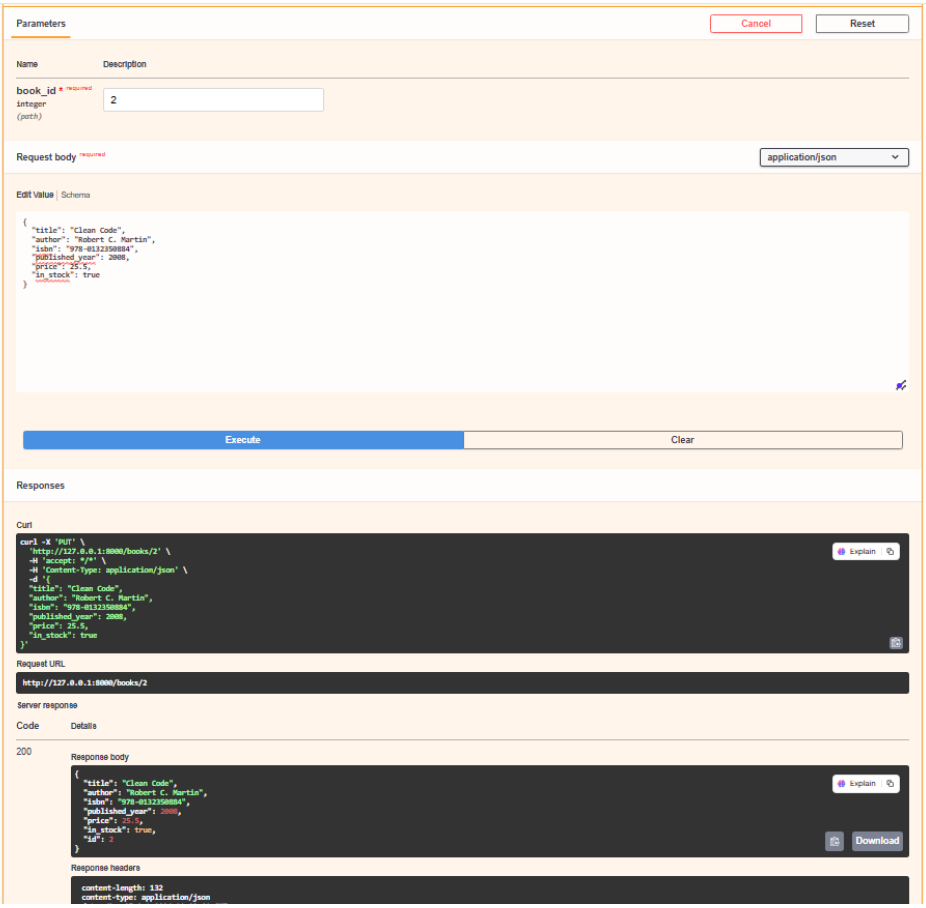
Expected result: 200 OK with the book details.



4. PUT /books/{id} – Update a book

The price of book 2 was changed to 25.5.

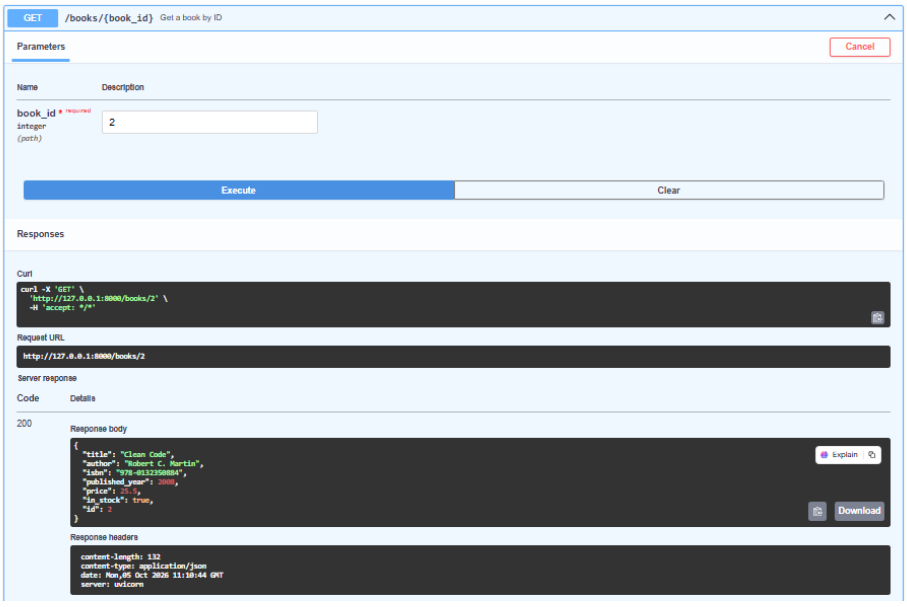
Expected result: 200 OK with the updated price.



5. GET /books/{id} – Verify the update was saved

Book 2 was fetched again after the update.

Expected result: The price is now 25.5, which confirms the change was stored in the database.



6. DELETE /books/{id} – Delete a book

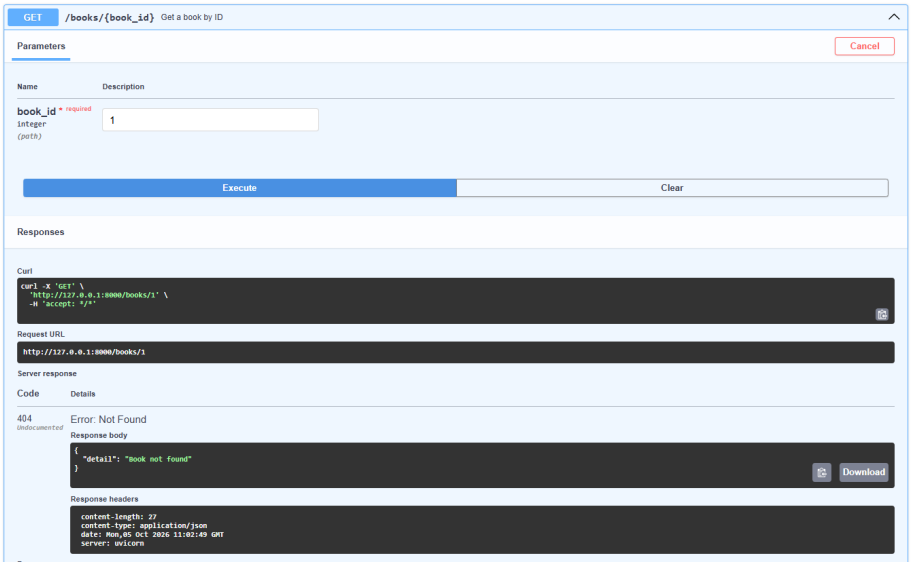
Book 1 was deleted.

Expected result: 204 No Content (success, empty response body). Response received: Status 204.

7. Error handling: book not found

Book 1 was requested again after it was deleted.

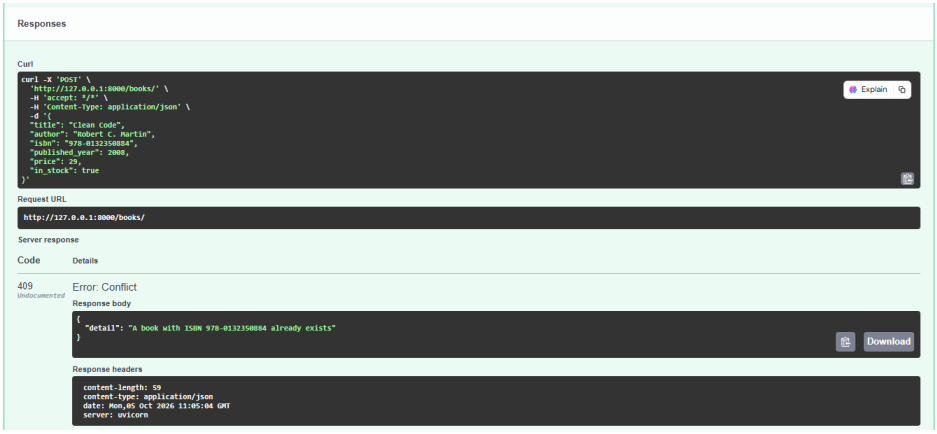
Expected result: 404 Not Found with the message "Book not found".



8. Error handling: duplicate ISBN

A POST was sent with an ISBN that already exists in the database.

Expected result: 409 Conflict with the message that the ISBN already exists.



7. Summary

Test	Status code	Result
POST – create book	201	Pass
GET – list books	200	Pass
GET – book by ID	200	Pass
PUT – update book	200	Pass
DELETE – delete book	204	Pass
GET – deleted book (not found)	404	Pass
POST – duplicate ISBN	409	Pass

All five CRUD operations work, and error cases return proper status codes and messages.